



BÀI TẬP LỚN THỰC TẬP CƠ SỞ

Tên đề tài:

XÂY DỰNG HỆ THỐNG GỢI Ý PHIM DỰA TRÊN COLLABORATIVE FILTERING (Ứng dụng trên bộ dữ liệu MovieLens ml-100k)

Họ và tên sinh viên: Mã sinh viên:

Nguyễn Văn Trường B23DCCN870

Giảng viên hướng dẫn: TS. Kim Ngọc Bách

HÀ NỘI, 06/2026

LỜI CẢM ƠN

Trước tiên, tôi xin gửi lời cảm ơn chân thành đến giảng viên hướng dẫn đã tận tình định hướng và góp ý trong suốt quá trình thực hiện bài tập lớn này.

Tôi xin cảm ơn Học viện Công nghệ Bưu chính Viễn thông và Khoa Công nghệ Thông tin 1 đã tạo điều kiện học tập, cung cấp nền tảng kiến thức vững chắc về Trí tuệ Nhân tạo và Học máy.

Tôi cũng xin cảm ơn nhóm GroupLens Research (Đại học Minnesota) đã công khai bộ dữ liệu `Movie-Lens ml-100k` phục vụ nghiên cứu học thuật, và cộng đồng mã nguồn mở đã xây dựng các thư viện NumPy, Pandas, FastAPI mà đề tài này sử dụng.

Cuối cùng, tôi xin cảm ơn gia đình và bạn bè đã động viên, hỗ trợ tôi trong suốt quá trình học tập và hoàn thành đề tài.

Hà Nội, tháng 6 năm 2026

Sinh viên thực hiện

Nguyễn Văn Trường

Mã sinh viên: B23DCCN870

MỤC LỤC

DANH MỤC HÌNH VẼ.....	iv
DANH MỤC BẢNG BIỂU	v
DANH MỤC THUẬT NGỮ VÀ TỪ VIẾT TẮT.....	vi
CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI.....	1
1.1 Đặt vấn đề.....	1
1.2 Mục tiêu đề tài	1
1.3 Phạm vi nghiên cứu	2
1.4 Bố cục bài tập lớn	2
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT.....	3
2.1 Tổng quan về Hệ thống Gợi ý.....	3
2.1.1 Định nghĩa	3
2.1.2 Phân loại	3
2.2 Collaborative Filtering	3
2.2.1 Nguyên lý hoạt động	3
2.2.2 User-based Collaborative Filtering	3
2.2.3 Item-based Collaborative Filtering	4
2.3 Matrix Factorization.....	4
2.3.1 Ý tưởng cơ bản.....	4
2.3.2 Hàm mất mát và Regularization	5
2.3.3 Stochastic Gradient Descent và theo dõi hội tụ	5
2.4 Các chỉ số đánh giá.....	5
2.4.1 Chiến lược chia dữ liệu Train/Test	5
2.4.2 MAE và RMSE.....	5
2.4.3 Precision@K và Recall@K	6
2.5 Bộ dữ liệu MovieLens ml-100k.....	6
2.6 Công nghệ và phương pháp triển khai.....	6
2.6.1 Triển khai thuật toán.....	6
2.6.2 Stack công nghệ	7

CHƯƠNG 3. THỰC NGHIỆM VÀ ĐÁNH GIÁ KẾT QUẢ	8
3.1 Quy trình tổng thể	8
3.2 Thu thập và tiền xử lý dữ liệu	8
3.2.1 Nguồn dữ liệu	8
3.2.2 Tiền xử lý	8
3.2.3 Phân tích khám phá dữ liệu (EDA)	8
3.3 Thiết kế kiến trúc hệ thống	9
3.3.1 Kiến trúc tổng thể	9
3.3.2 Thiết kế API	10
3.3.3 Cấu trúc dự án	10
3.4 Chiến lược chia tập Train/Test	10
3.5 Môi trường thực nghiệm	10
3.6 Cài đặt chi tiết các mô hình	11
3.6.1 User-based Collaborative Filtering	11
3.6.2 Item-based Collaborative Filtering	11
3.6.3 Matrix Factorization (Funk SVD)	11
3.7 Kết quả đánh giá mô hình	11
3.7.1 MAE và RMSE	11
3.7.2 Precision@K và Recall@K	12
3.7.3 Điều chỉnh siêu tham số và đường cong hội tụ	12
3.8 Ví dụ kết quả gợi ý	13
3.9 Giao diện demo hệ thống	13
3.9.1 Hướng dẫn cài đặt và chạy	13
3.9.2 Mô tả giao diện	13
3.9.3 Kiểm tra hoạt động toàn hệ thống	14
CHƯƠNG 4. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....	15
4.1 Kết quả đạt được	15
4.1.1 Về lý thuyết	15
4.1.2 Về kỹ thuật	15
4.1.3 Về kết quả thực nghiệm	15
4.2 Hạn chế của đề tài	15

4.3 Hướng phát triển tương lai	16
TÀI LIỆU THAM KHẢO	17
PHỤ LỤC	19
A. MÃ NGUỒN HỆ THỐNG CINEREC.....	19
A.1 File models.py — Lỗi thuật toán.....	19
A.2 File app.py — FastAPI Backend	25
A.3 File run.py — Script khởi động	26

DANH MỤC HÌNH VẼ

Hình 3.1	Phân phối <i>rating</i> trên bộ dữ liệu MovieLens ml-100k ($n = 100.000$). Phân phối lệch trái với <i>rating</i> 4 chiếm tỷ lệ cao nhất.	9
Hình 3.2	Kiến trúc ba lớp của hệ thống CineRec và luồng dữ liệu khi xử lý một yêu cầu gợi ý .	9
Hình 3.3	<i>Temporal split</i> theo từng người dùng: 80% <i>rating</i> sớm nhất làm <i>train</i> , 20% muộn nhất làm <i>test</i> . Mỗi người dùng được chia độc lập theo mốc thời gian của chính họ.	10
Hình 3.4	Đường cong RMSE theo <i>epoch</i> của mô hình <i>Matrix Factorization</i> ($K = 20$, $\alpha = 0,005$, $\lambda = 0,02$). <i>Train</i> và <i>Validation</i> hội tụ song song, không có dấu hiệu <i>overfitting</i>	12
Hình 3.5	Tổng quan giao diện CineRec (<i>dark theme</i>): header thống kê hệ thống, bảng điều khiển bên trái (nhập User ID, chọn phương pháp, điều chỉnh số gợi ý), panel kết quả bên phải.	13
Hình 3.6	Kết quả gợi ý <i>Top-10</i> cho User 1 (MF SVD) hiển thị dạng card với thanh điểm dự đoán và số sao trực quan (khớp với Bảng 3.7), kèm lịch sử xem với màu sắc phân biệt mức <i>rating</i> (xanh lá = cao, đỏ = thấp).	14

DANH MỤC BẢNG BIỂU

Bảng 2.1	Thống kê tổng quan dataset MovieLens ml-100k	6
Bảng 2.2	Các công nghệ sử dụng trong hệ thống CineRec	7
Bảng 3.1	Danh sách API endpoints	10
Bảng 3.2	Kết quả chia tập train/test (temporal split 80/20)	10
Bảng 3.3	Môi trường thực nghiệm	10
Bảng 3.4	Kết quả MAE và RMSE trên tập <i>test</i> (temporal split 80/20)	11
Bảng 3.5	Kết quả Precision@10 và Recall@10 (tập con 200 users, ngưỡng <i>relevant</i> = rating ≥ 4)	12
Bảng 3.6	Kết quả điều chỉnh siêu tham số MF	12
Bảng 3.7	Top-10 gợi ý cho User 1 (Matrix Factorization, $K = 20$)	13
Bảng 4.1	Tổng hợp kết quả thực nghiệm trên MovieLens ml-100k	15

DANH MỤC THUẬT NGỮ VÀ TỪ VIẾT TẮT

Thuật ngữ	Ý nghĩa
ALS	Alternating Least Squares — thuật toán tối ưu hóa xen kẽ cho Matrix Factorization trên dữ liệu ngầm định
ANN	Approximate Nearest Neighbors — tìm kiếm láng giềng gần đúng hiệu quả cho không gian chiều cao
API	Application Programming Interface — Giao diện lập trình ứng dụng
ASGI	Asynchronous Server Gateway Interface — chuẩn giao tiếp máy chủ bất đồng bộ cho Python
BPR	Bayesian Personalized Ranking — phương pháp huấn luyện xếp hạng cá nhân hóa cho dữ liệu ngầm định
CF	Collaborative Filtering — Lọc cộng tác, phương pháp gợi ý dựa trên hành vi của cộng đồng người dùng
CSV	Comma-Separated Values — định dạng file dữ liệu phân cách bằng dấu phẩy
EDA	Exploratory Data Analysis — Phân tích khám phá dữ liệu
GD	Gradient Descent — thuật toán tối ưu hóa theo hướng gradient giảm dần
HTML	HyperText Markup Language — Ngôn ngữ đánh dấu siêu văn bản, dùng xây dựng giao diện web
HTTP	HyperText Transfer Protocol — giao thức truyền siêu văn bản, nền tảng của World Wide Web
JSON	JavaScript Object Notation — định dạng trao đổi dữ liệu nhẹ, dễ đọc cho cả người và máy
MAE	Mean Absolute Error — Sai số tuyệt đối trung bình, chỉ số đánh giá độ chính xác dự đoán rating
MF	Matrix Factorization — Phân rã ma trận, kỹ thuật Model-based Collaborative Filtering
MRR	Mean Reciprocal Rank — chỉ số đánh giá chất lượng xếp hạng trong hệ thống gợi ý
NCF	Neural Collaborative Filtering — hệ thống gợi ý kết hợp mạng neural với Collaborative Filtering
NDCG	Normalized Discounted Cumulative Gain — chỉ số đánh giá chất lượng xếp hạng có tính đến vị trí trong danh sách
RAM	Random Access Memory — bộ nhớ truy cập ngẫu nhiên
REST	Representational State Transfer — kiến trúc thiết kế API web theo nguyên tắc không trạng thái
RMSE	Root Mean Squared Error — Căn bậc hai của sai số bình phương trung bình, chỉ số đánh giá độ chính xác dự đoán rating

Thuật ngữ	Ý nghĩa
SGD	Stochastic Gradient Descent — Hạ gradient ngẫu nhiên, cập nhật tham số sau mỗi mẫu dữ liệu đơn lẻ
SVD	Singular Value Decomposition — Phân rã giá trị suy biến, nền tảng của các thuật toán Matrix Factorization
UI	User Interface — Giao diện người dùng

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

1.1 Đặt vấn đề

Trong bối cảnh bùng nổ thông tin và sự phát triển mạnh mẽ của thương mại điện tử, người dùng ngày càng phải đối mặt với hiện tượng quá tải thông tin (*information overload*). Với số lượng sản phẩm lớn, hàng chục nghìn bộ phim hay hàng tỷ bài hát được cung cấp trực tuyến, việc tìm kiếm nội dung phù hợp với sở hữu cá nhân trở nên phức tạp và tiêu tốn nhiều thời gian của người sử dụng.

Hệ thống gợi ý (*Recommendation System*) là giải pháp công nghệ hiệu quả cho vấn đề trên. Đây là một dạng hệ thống lọc thông tin có khả năng dự đoán sở thích, đánh giá hoặc hành vi của người dùng đối với một tập các sản phẩm hoặc dịch vụ, từ đó đề xuất những hạng mục phù hợp nhất. Việc tối ưu hóa cơ chế gợi ý giúp các nền tảng số gia tăng trải nghiệm người dùng, tối ưu hóa tỷ lệ giữ chân khách hàng và nâng cao hiệu quả kinh doanh. Hiện nay, hệ thống gợi ý đã đóng vai trò lõi tại những nền tảng lớn như Netflix, Amazon, Spotify và YouTube.

Đề án này tập trung nghiên cứu và triển khai một hệ thống gợi ý phim hoàn chỉnh — được đặt tên là **CineRec** — dựa trên kỹ thuật Lọc cộng tác (*Collaborative Filtering*) và Phân rã ma trận (*Matrix Factorization*), áp dụng trên bộ dữ liệu thực tế MovieLens ml-100k do nhóm GroupLens Research (Đại học Minnesota) cung cấp. Bộ dữ liệu này được lựa chọn dựa trên hai lý do cốt lõi:

1. Đây là tập dữ liệu chuẩn (*benchmark*) được cộng đồng nghiên cứu học thuật công nhận rộng rãi, cho phép đối chiếu kết quả thực nghiệm một cách khách quan với các công trình liên quan.
2. Với quy mô 100.000 lượt đánh giá từ 943 người dùng cho 1.682 bộ phim, dữ liệu đủ lớn để kiểm chứng thuật toán một cách thực chất nhưng không gây quá tải tài nguyên hệ thống khi triển khai trên máy tính cá nhân — phù hợp với định hướng tự cài đặt thuật toán từ đầu của đề tài.

1.2 Mục tiêu đề tài

Mục tiêu lý thuyết: Nắm vững nền tảng lý thuyết của hệ thống gợi ý, đặc biệt là kỹ thuật *Collaborative Filtering* và *Matrix Factorization* (Funk SVD). Hiểu rõ các chỉ số đánh giá phổ biến: *MAE*, *RMSE*, *Precision@K*, *Recall@K*.

Mục tiêu kỹ thuật: Triển khai thành công ba mô hình gợi ý (*User-based CF*, *Item-based CF*, *Matrix Factorization* với SGD) bằng Python, tự cài đặt phần cốt lõi của thuật toán mà không sử dụng thư viện ML chuyên dụng như Scikit-learn hay Surprise. Các thư viện tính toán NumPy và Pandas vẫn được sử dụng để tối ưu hóa hiệu năng thông qua *vectorization* (phép nhân ma trận, tính *cosine similarity* hàng loạt), thay thế cho vòng lặp Python thuần túy vốn không phù hợp với ma trận kích thước 943×1.682 .

Mục tiêu thực nghiệm: Huấn luyện, đánh giá và so sánh hiệu suất các mô hình trên tập dữ liệu ml-100k theo chiến lược *temporal split* (80/20). Thực hiện điều chỉnh siêu tham số để tối ưu kết quả, hướng đến đạt các chỉ số *MAE* và *RMSE* tương đương hoặc tiệm cận với kết quả tham chiếu của các nghiên cứu tiêu chuẩn trên cùng bộ dữ liệu. Bên cạnh đó, đánh giá chất lượng gợi ý thực tế qua *Precision@10* và *Recall@10*.

Mục tiêu ứng dụng: Xây dựng hệ thống demo hoàn chỉnh gồm backend FastAPI và frontend HTML5/CSS3/JavaScript thuần, cho phép người dùng nhập User ID (1–943), chọn phương pháp gợi ý và nhận danh sách Top-*N* phim cùng lịch sử xem. Hệ thống đảm bảo thời gian phản hồi API (*response time*) dưới 1 giây sau khi khởi động, minh chứng khả năng ứng dụng thực tiễn.

1.3 Phạm vi nghiên cứu

Để giải quyết các nhiệm vụ xác định tại mục 1.2, đề tài đề xuất định hướng xây dựng và giới hạn bài toán theo các tiêu chí kỹ thuật cụ thể dưới đây:

Bài toán: Dự đoán điểm đánh giá (*rating prediction*) và gợi ý Top- N phim (*item recommendation*). Hệ thống hỗ trợ cả hai chức năng: trả về điểm dự đoán (*predicted rating*) và lịch sử xem của từng người dùng.

Dữ liệu: Sử dụng bộ dữ liệu công khai MovieLens ml-100k (độ thưa thớt của ma trận khoảng 93,7%). Kích thước dữ liệu vừa đủ để kiểm chứng thuật toán từ đầu (*from scratch*) trên máy tính cá nhân không cần GPU, đồng thời là *benchmark* chuẩn trong nghiên cứu học thuật để so sánh kết quả. Đây là lý do ml-100k được ưu tiên so với các phiên bản lớn hơn như ml-1m hay ml-20m.

Phương pháp: Tập trung vào ba mô hình chính bao gồm:

- *User-based CF* với *Pearson/Cosine similarity* ($K = 30$ láng giềng).
- *Item-based CF* với *adjusted cosine similarity* ($K = 30$).
- *Matrix Factorization* theo thuật toán Funk SVD tối ưu bằng SGD ($K = 20$ nhân tố ẩn, 30 *epochs*).

Đề tài không đi sâu nghiên cứu kỹ thuật *Content-based Filtering* hay Học sâu (*Deep Learning*).

Ngôn ngữ và công nghệ: Sử dụng ngôn ngữ Python 3.x; các thư viện NumPy và Pandas cho tính toán ma trận; FastAPI đảm nhiệm vai trò backend REST API; giao diện frontend được xây dựng bằng HTML5/CSS3/JavaScript thuần (không sử dụng framework). Hệ thống khởi động server thông qua Uvicorn và phục vụ giao diện web qua tệp `index.html`.

Môi trường triển khai: Thực hiện hoàn toàn trên máy tính cá nhân, không yêu cầu GPU hay hạ tầng đám mây (*cloud infrastructure*). Toàn bộ quá trình huấn luyện cả ba mô hình và khởi động hệ thống được hoàn thành chỉ trong vòng vài chục giây, chứng minh tính khả thi và hiệu quả của giải pháp trên các tài nguyên phần cứng thông thường.

1.4 Bố cục bài tập lớn

Phần còn lại của báo cáo bài tập lớn này được tổ chức và cấu trúc chặt chẽ nhằm làm rõ toàn bộ quy trình nghiên cứu, triển khai hệ thống.

Tiếp sau phần mở đầu, Chương 2 sẽ tập trung làm sáng tỏ cơ sở lý thuyết nền tảng của hệ thống gợi ý, đi sâu vào bản chất toán học của kỹ thuật Lọc cộng tác (*Collaborative Filtering*), phương pháp Phân rã ma trận (*Matrix Factorization* với thuật toán Funk SVD), đồng thời định nghĩa chi tiết các chỉ số kiểm định sai số và chất lượng mô hình.

Trong Chương 3, tôi giới thiệu toàn bộ quy trình phân tích đặc trưng của bộ dữ liệu thực nghiệm ml-100k, từ đó phác thảo kiến trúc hệ thống tổng thể kết hợp giữa backend FastAPI và frontend trực quan, làm tiền đề cho chiến lược xây dựng ba mô hình cốt lõi.

Toàn bộ quá trình hiện thực hóa thuật toán từ đầu (*from scratch*), kết quả thực nghiệm chi tiết theo các chỉ số *MAE*, *RMSE*, *Precision@10*, *Recall@10* cũng như các đánh giá, so sánh định lượng và giao diện demo của hệ thống **CineRec** sẽ được trình bày cụ thể trong Chương 4.

Cuối cùng, Chương 5 đóng vai trò tổng kết lại toàn bộ các kết quả thực tiễn đã đạt được, thẳng thắn nhìn nhận những điểm hạn chế còn tồn tại và đề xuất định hướng phát triển khả thi trong tương lai.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1 Tổng quan về Hệ thống Gợi ý

2.1.1 Định nghĩa

Hệ thống gợi ý (*Recommender System* hoặc *Recommendation System*) là một dạng hệ thống lọc thông tin có nhiệm vụ dự đoán xếp hạng (*rating*) hoặc sở thích mà một người dùng cụ thể sẽ dành cho một mục (*item*) nhất định, từ đó đưa ra các gợi ý phù hợp nhất. Khái niệm này được hình thành từ những năm 1990 với hệ thống lọc cộng tác GroupLens [1], và kể từ đó đã trở thành một lĩnh vực nghiên cứu sôi động trong cộng đồng học máy và khai phá dữ liệu.

2.1.2 Phân loại

Dựa trên phương pháp tiếp cận, hệ thống gợi ý được chia thành ba nhóm chính [2]:

- **Content-based Filtering** (Lọc theo nội dung): Gợi ý các mục có nội dung/đặc trưng tương tự với những mục người dùng đã thích trong quá khứ. Ưu điểm: không cần dữ liệu từ người dùng khác. Nhược điểm: bị giới hạn trong vùng nội dung quen thuộc (*over-specialization*). Phương pháp này không được sử dụng trong đề án.
- **Collaborative Filtering** (Lọc cộng tác): Gợi ý dựa trên hành vi/đánh giá của cộng đồng người dùng. Không cần biết nội dung sản phẩm, chỉ cần ma trận tương tác *user-item*. Đây là phương pháp được sử dụng chủ yếu trong đề án này.
- **Hybrid Approach** (Kết hợp): Kết hợp cả hai phương pháp trên để tận dụng ưu điểm và khắc phục nhược điểm của từng loại. Được ứng dụng bởi các hệ thống lớn như Netflix, YouTube.

2.2 Collaborative Filtering

2.2.1 Nguyên lý hoạt động

Collaborative Filtering (CF) dựa trên giả định cơ bản: những người dùng có hành vi tương tự trong quá khứ sẽ có sở thích tương tự trong tương lai [3]. CF khai thác ma trận tương tác *user-item* R (kích thước $n \times m$) trong đó $R[u][i]$ là *rating* mà người dùng u dành cho sản phẩm i . Các ô chưa có giá trị (NaN) là những *rating* cần được dự đoán.

CF được chia thành hai nhánh chính: **Memory-based CF** (dựa trên bộ nhớ, gồm *User-based* và *Item-based*) và **Model-based CF** (dựa trên mô hình, gồm *Matrix Factorization* và các mô hình học sâu). Đề án này triển khai cả ba phương pháp: *UserBasedCF*, *ItemBasedCF* và *MatrixFactorization*, được cài đặt hoàn toàn từ đầu bằng NumPy (không sử dụng thư viện CF chuyên dụng).

Vấn đề Cold Start: CF gặp khó khăn khi người dùng mới hoặc sản phẩm mới chưa có đủ dữ liệu tương tác — không thể tính được *similarity* vì vector *rating* gần như rỗng. Trong hệ thống **CineRec**, trường hợp này được xử lý bằng cách *fallback* về *rating* trung bình của người dùng ($\bar{r}_u$) khi không tìm được láng giềng hợp lệ (*similarity* > 0).

2.2.2 User-based Collaborative Filtering

User-based CF tìm kiếm K người dùng có sở thích tương đồng nhất với người dùng mục tiêu u (K *nearest neighbors*), sau đó tổng hợp *rating* của họ để dự đoán *rating* cho sản phẩm i chưa được đánh giá. Trong đề án này, $K = 30$.

Độ tương đồng giữa hai người dùng được đo bằng **mean-centered cosine similarity** — trừ *rating* trung bình của từng người dùng để loại bỏ *bias* đánh giá (xu hướng cho điểm cao hoặc thấp hơn trung bình):

$$\text{sim}(u, v) = \frac{\sum_i (r_{ui} - \bar{r}_u)(r_{vi} - \bar{r}_v)}{\|r_u - \bar{r}_u\| \cdot \|r_v - \bar{r}_v\|} \quad (2.1)$$

Công thức dự đoán *rating* có hiệu chỉnh *bias*:

$$\hat{r}_{ui} = \bar{r}_u + \frac{\sum_{v \in N(u)} \text{sim}(u, v) \cdot (r_{vi} - \bar{r}_v)}{\sum_{v \in N(u)} |\text{sim}(u, v)|} \quad (2.2)$$

Trong đó $\bar{r}_u$ là *rating* trung bình của user u , $N(u)$ là tập K láng giềng gần nhất của u đã đánh giá sản phẩm i . Kết quả được giới hạn trong khoảng $[1, 5]$ bằng `np.clip`. Toàn bộ ma trận *similarity* (943×943) được tính trước trong bước `fit()` bằng phép nhân ma trận NumPy ($R_n \cdot R_n^T$), đảm bảo hiệu năng khi phục vụ nhiều yêu cầu gợi ý.

2.2.3 Item-based Collaborative Filtering

Item-based CF tính độ tương đồng giữa các sản phẩm thay vì giữa người dùng [3]. Phương pháp sử dụng **adjusted cosine similarity** — trừ *rating* trung bình của người dùng (không phải của sản phẩm) để loại bỏ *bias* đánh giá cá nhân khi so sánh hai sản phẩm:

$$\text{sim}(i, j) = \frac{\sum_u (r_{ui} - \bar{r}_u)(r_{uj} - \bar{r}_u)}{\|r_i - \bar{r}\| \cdot \|r_j - \bar{r}\|} \quad (2.3)$$

Công thức dự đoán *rating* cho user u và item i , dựa trên $K = 30$ sản phẩm tương đồng nhất mà user u đã đánh giá:

$$\hat{r}_{ui} = \frac{\sum_{j \in N(i, u)} \text{sim}(i, j) \cdot r_{uj}}{\sum_{j \in N(i, u)} |\text{sim}(i, j)|} \quad (2.4)$$

Công thức này sử dụng trực tiếp *rating* thô r_{uj} vì *adjusted cosine similarity* đã hấp thụ *bias* người dùng vào bước tính *similarity*. Ma trận *similarity* item (1.682×1.682) cũng được tính trước trong `fit()` để phục vụ dự đoán *online* nhanh.

Ưu điểm so với User-based CF: Ma trận tương đồng item ổn định hơn theo thời gian (danh sách phim ít thay đổi hơn người dùng) và có thể tính toán *offline*, phục vụ dự đoán *online* nhanh hơn.

2.3 Matrix Factorization

2.3.1 Ý tưởng cơ bản

Matrix Factorization (MF) là phương pháp thuộc nhóm *Model-based CF*, nổi bật qua cuộc thi Netflix Prize (2009) [4]. Ý tưởng cốt lõi là phân rã ma trận *rating* R ($n \times m$) thành tích của hai ma trận *latent factor* có chiều thấp hơn:

$$R \approx P \times Q^T \quad (2.5)$$

Trong đó: P ($n \times K$) là ma trận *latent factor* của người dùng; Q ($m \times K$) là ma trận *latent factor* của sản phẩm; K là số *latent factors* (trong đề án $K = 20$). *Rating* được dự đoán bởi tích vô hướng $\hat{r}_{ui} = p_u \cdot q_i$.

Các *latent factor* không có nhãn tường minh, nhưng ngầm học các đặc trưng ẩn của dữ liệu — ví dụ: thể

loại phim, phong cách đạo diễn, mức độ kịch tính. Người dùng và phim được ánh xạ vào cùng một không gian K chiều, và *rating* được ước lượng qua mức độ “khớp” giữa hai vector.

Mô hình **Funk SVD** [5] mở rộng thêm *bias* để tăng độ chính xác:

$$\hat{r}_{ui} = \mu + b_u + b_i + p_u \cdot q_i \quad (2.6)$$

Trong đó μ là *rating* trung bình toàn cục, b_u là *bias* của user u (xu hướng cho điểm cao/thấp hơn trung bình), b_i là *bias* của item i (phim được yêu thích hay không nói chung).

2.3.2 Hàm mất mát và Regularization

Mô hình tối thiểu hóa hàm mất mát bình phương có thêm *regularization* L2 để tránh *overfitting*:

$$L = \sum_{(u,i) \in \text{rated}} (r_{ui} - \hat{r}_{ui})^2 + \lambda \cdot (\|p_u\|^2 + \|q_i\|^2 + b_u^2 + b_i^2) \quad (2.7)$$

Regularization với hệ số $\lambda = 0,02$ phạt các giá trị tham số quá lớn, giúp mô hình tổng quát hóa tốt hơn trên dữ liệu chưa thấy. Không có *regularization*, mô hình dễ ghi nhớ tập *train* mà không dự đoán được tập *test*.

2.3.3 Stochastic Gradient Descent và theo dõi hội tụ

Thuật toán *Stochastic Gradient Descent* (SGD) cập nhật tham số sau mỗi *rating* đơn lẻ (thay vì toàn bộ dataset như Batch GD), giúp hội tụ nhanh hơn [4]. Với *residual* $e_{ui} = r_{ui} - \hat{r}_{ui}$ và *learning rate* $\alpha = 0,005$, quy tắc cập nhật:

$$p_u \leftarrow p_u + \alpha \cdot (e_{ui} \cdot q_i - \lambda \cdot p_u) \quad (2.8)$$

$$q_i \leftarrow q_i + \alpha \cdot (e_{ui} \cdot p_u - \lambda \cdot q_i) \quad (2.9)$$

$$b_u \leftarrow b_u + \alpha \cdot (e_{ui} - \lambda \cdot b_u) \quad (2.10)$$

$$b_i \leftarrow b_i + \alpha \cdot (e_{ui} - \lambda \cdot b_i) \quad (2.11)$$

Dữ liệu được xáo trộn ngẫu nhiên (`np.random.shuffle`) đầu mỗi *epoch* để tránh *bias* thứ tự. Ma trận P và Q được khởi tạo từ phân phối chuẩn $\mathcal{N}(0, 0,1)$; *bias* b_u, b_i khởi tạo bằng 0. Mô hình chạy 30 *epochs*. Sau mỗi *epoch*, RMSE trên cả tập *train* và tập *validation* được ghi lại để theo dõi hội tụ và phát hiện *overfitting*; hai đường *loss curve* này được hiển thị trực quan trên giao diện web **CineRec**.

2.4 Các chỉ số đánh giá

2.4.1 Chiến lược chia dữ liệu Train/Test

Dữ liệu được chia theo chiến lược **temporal split**: với mỗi người dùng, 20% *rating* có *timestamp* muộn nhất được giữ làm tập *test*, 80% còn lại làm tập *train*. Chiến lược này phản ánh đúng kịch bản thực tế — mô hình chỉ học từ lịch sử quá khứ và dự đoán hành vi tương lai — tránh hiện tượng *data leakage* mà *random split* có thể gây ra (mô hình vô tình “nhìn thấy tương lai” khi huấn luyện).

2.4.2 MAE và RMSE

Hai chỉ số đánh giá chất lượng dự đoán *rating* trên tập *test* [6]:

$$\text{MAE} = \frac{1}{n} \sum_{(u,i)} |r_{ui} - \hat{r}_{ui}| \quad (2.12)$$

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{(u,i)} (r_{ui} - \hat{r}_{ui})^2} \quad (2.13)$$

MAE đo sai số tuyệt đối trung bình, dễ hiểu và ít nhạy với *outlier*. RMSE phạt nặng hơn các sai số lớn do bình phương hóa. Cả hai giá trị càng nhỏ, mô hình dự đoán càng chính xác.

2.4.3 Precision@K và Recall@K

Hai chỉ số đánh giá chất lượng danh sách gợi ý Top- K , sử dụng ngưỡng *rating* $\geq 4,0$ để xác định sản phẩm “relevant” (khớp với tham số `threshold=4.0` trong hàm `precision_recall_at_k()`):

$$\text{Precision@K} = \frac{|\{\text{relevant items}\} \cap \{\text{top-}K \text{ recs}\}|}{K} \quad (2.14)$$

$$\text{Recall@K} = \frac{|\{\text{relevant items}\} \cap \{\text{top-}K \text{ recs}\}|}{|\{\text{relevant items}\}|} \quad (2.15)$$

Precision@K đo tỷ lệ gợi ý chính xác trong Top- K ; Recall@K đo tỷ lệ sản phẩm *relevant* được tìm thấy trong Top- K . Hai chỉ số này bổ sung cho MAE/RMSE vì đánh giá trực tiếp chất lượng danh sách gợi ý thay vì độ chính xác từng *rating* đơn lẻ. Do chi phí tính toán, Precision@10 và Recall@10 được tính trên *subset* 200 người dùng đầu tiên của tập *test*.

2.5 Bộ dữ liệu MovieLens ml-100k

MovieLens ml-100k là bộ dữ liệu chuẩn (*benchmark*) được phát hành bởi nhóm GroupLens Research, Đại học Minnesota [7]. Đây là một trong những bộ dữ liệu được sử dụng rộng rãi nhất trong nghiên cứu hệ thống gợi ý nhờ kích thước vừa phải, chất lượng tốt và tính khả dụng công khai.

Bảng 2.1: Thống kê tổng quan dataset MovieLens ml-100k

Thuộc tính	Giá trị	Ghi chú
Số người dùng	943	Đã ẩn danh
Số phim	1.682	Nhiều thể loại
Tổng số rating	100.000	Thang điểm 1–5 (nguyên)
Rating trung bình	3,53	Lệch về phía cao
Sparsity	$\approx 93,70\%$	$1 - \frac{100.000}{943 \times 1.682}$, tính trên toàn bộ ma trận trước khi chia train/test
Thời gian	1997–1998	Unix timestamp

Ma trận *user-item* có độ thưa thớt cao ($\approx 93,70\%$), nghĩa là trung bình mỗi người dùng chỉ đánh giá khoảng 6,3% số phim có trong hệ thống. Đây là thách thức điển hình của *Collaborative Filtering*, đòi hỏi thuật toán phải xử lý tốt với dữ liệu *sparse*.

2.6 Công nghệ và phương pháp triển khai

2.6.1 Triển khai thuật toán

Toàn bộ ba mô hình *Collaborative Filtering* (User-based CF, Item-based CF, Matrix Factorization) được **triển khai hoàn toàn từ đầu** (*from scratch*) bằng NumPy, không sử dụng thư viện học máy chuyên dụng như Surprise, LightFM hay các mô hình có sẵn của `scikit-learn`. Điều này đảm bảo:

- Hiểu sâu cơ chế hoạt động của từng thuật toán (công thức tính *similarity*, quy tắc dự đoán, cập nhật tham số SGD).
- Kiểm soát hoàn toàn luồng xử lý dữ liệu, *hyperparameter tuning* và tối ưu hóa hiệu năng.

- Tính toán ma trận tương đồng bằng phép nhân ma trận vectorized (NumPy matrix multiplication), thay thế vòng lặp Python thuần, giảm thời gian tính từ phút xuống giây.

2.6.2 Stack công nghệ

Bảng 2.2: Các công nghệ sử dụng trong hệ thống CineRec

Công nghệ	Phiên bản	Mục đích sử dụng
Python	3.10+	Ngôn ngữ lập trình chính
NumPy	1.24+	Lỗi thuật toán: Tính toán ma trận <i>similarity</i> , phép nhân ma trận ($R_n \cdot R_n^T$), SGD cập nhật tham số P, Q, b_u, b_i
Pandas	2.0+	Đọc dữ liệu CSV, <i>pivot table</i> (xây dựng ma trận R), <i>temporal split</i> , phân tích EDA
Scikit-learn	1.3+	Chỉ dùng cho đánh giá: <code>mean_absolute_error()</code> , <code>mean_squared_error()</code> để tính MAE/RMSE. Không sử dụng cho xây dựng mô hình CF hay MF
FastAPI	0.110+	Web framework backend RESTful API (endpoint <code>/api/recommend</code> , <code>/api/stats</code>)
Uvicorn	0.27+	ASGI server chạy FastAPI (port 8000)
HTML5/CSS3/JS	—	Giao diện web frontend SPA (không dùng React/Vue, thuần Vanilla JS)

Tóm tắt kiến trúc:

- **Data Layer:** Pandas đọc `ratings.csv`, `movies.csv`; `temporal_split()` chia train/test.
- **Model Layer:** Ba lớp `UserBasedCF`, `ItemBasedCF`, `MatrixFactorization` cài đặt từ đầu bằng NumPy.
- **API Layer:** FastAPI expose endpoint RESTful, trả JSON; Uvicorn phục vụ HTTP.
- **Presentation Layer:** HTML/CSS/JS render giao diện, gọi API qua `fetch()`, vẽ biểu đồ *loss curve* bằng Canvas API.

CHƯƠNG 3. THỰC NGHIỆM VÀ ĐÁNH GIÁ KẾT QUẢ

3.1 Quy trình tổng thể

Hệ thống **CineRec** được xây dựng theo quy trình năm giai đoạn tuần tự, đảm bảo tính logic và khả năng tái lập:

1. **Nghiên cứu lý thuyết (Giai đoạn 1):** Tìm hiểu tổng quan về *Recommender Systems*, *Collaborative Filtering*, *Matrix Factorization* và các chỉ số đánh giá (xem Chương 2).
2. **Thu thập và tiền xử lý dữ liệu (Giai đoạn 2):** Lựa chọn dataset, làm sạch dữ liệu, phân tích khám phá (EDA) và xây dựng ma trận *user-item*.
3. **Xây dựng mô hình (Giai đoạn 3):** Triển khai ba mô hình CF, huấn luyện bằng *Stochastic Gradient Descent*, chia tập *train/test*.
4. **Đánh giá và tối ưu (Giai đoạn 4):** Tính RMSE, MAE, Precision@K, Recall@K. Điều chỉnh siêu tham số.
5. **Xây dựng demo (Giai đoạn 5):** Xây dựng API backend và giao diện web, kiểm tra toàn hệ thống.

3.2 Thu thập và tiền xử lý dữ liệu

3.2.1 Nguồn dữ liệu

Dữ liệu được lấy từ bộ dữ liệu MovieLens *ml-100k* (<https://grouplens.org/datasets/movielens/100k/>) [7]. Bộ dữ liệu gồm hai file gốc chính:

- *u.data*: Chứa 100.000 dòng dữ liệu *rating* theo định dạng *tab-separated*: *userId*, *movieId*, *rating*, *timestamp*.
- *u.item*: Chứa thông tin 1.682 phim theo định dạng *pipe-separated*: *movieId*, *title*, *release_date*, *imdb_url* và 19 cột thể loại nhị phân.

Hai file trên được parse thành *ratings.csv* và *movies.csv* để thuận tiện cho việc đọc bằng Pandas.

3.2.2 Tiền xử lý

Đọc và chuẩn hóa: Đọc *u.data* với *encoding latin-1*, tách 19 cột thể loại nhị phân thành chuỗi *genres*.

Kiểm tra dữ liệu: Xác nhận không có giá trị thiếu (NaN) trong cột *rating*. Kiểm tra phân phối *rating* (nguyên, thuộc thang 1–5).

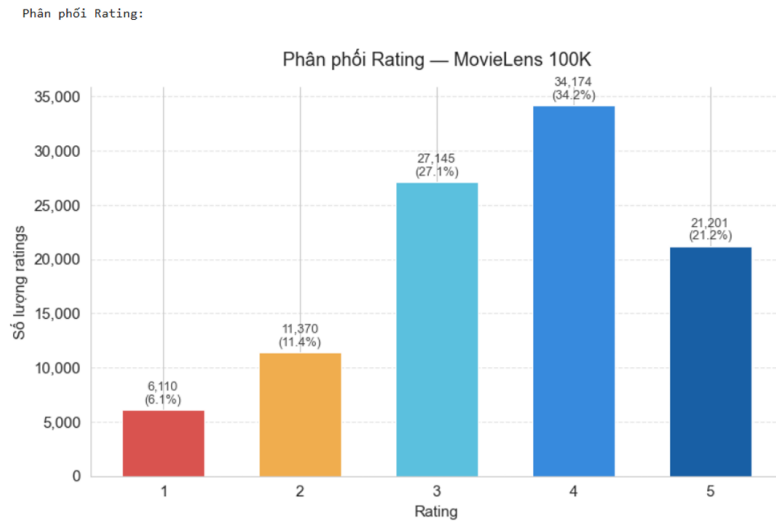
Xây dựng ma trận *user-item*: Tạo *pivot table* (943×1.615) với *userId* làm *index*, *movieId* làm cột. Các ô chưa có *rating* điền bằng 0. Kích thước cột là 1.615 (nhỏ hơn 1.682) vì tập *train* chỉ chứa phim xuất hiện trong 80% *rating* đầu tiên của mỗi người dùng.

Chia tập *train/test*: Áp dụng *temporal split* (chi tiết tại mục 3.4).

3.2.3 Phân tích khám phá dữ liệu (EDA)

Phân phối *rating*

Hình 3.1 thể hiện phân phối các mức *rating*. *Rating* 4 chiếm tỷ lệ cao nhất ($\approx 28\%$), tiếp theo là *rating* 3 ($\approx 27\%$) và 5 ($\approx 21\%$). *Rating* 1 ít nhất ($\approx 6\%$). Phân phối lệch trái — người dùng có xu hướng đánh giá cao, phản ánh *positive bias* điển hình trong dữ liệu *rating* tự nguyện.



Hình 3.1: Phân phối *rating* trên bộ dữ liệu MovieLens ml-100k ($n = 100.000$). Phân phối lệch trái với *rating* 4 chiếm tỷ lệ cao nhất.

Hoạt động người dùng và độ phổ biến phim

Phân phối số *rating* theo người dùng rất lệch (*long-tail distribution*): một số “super users” đánh giá hơn 700 phim, trong khi nhiều người dùng chỉ đánh giá 20 phim (ngưỡng tối thiểu). Tương tự, một số phim kinh điển được đánh giá hàng trăm lần (*Star Wars*: 584 lượt). Hai đặc điểm *long-tail* này là nguyên nhân trực tiếp dẫn đến độ thưa thớt $\approx 93,70\%$ của ma trận *user-item*.

3.3 Thiết kế kiến trúc hệ thống

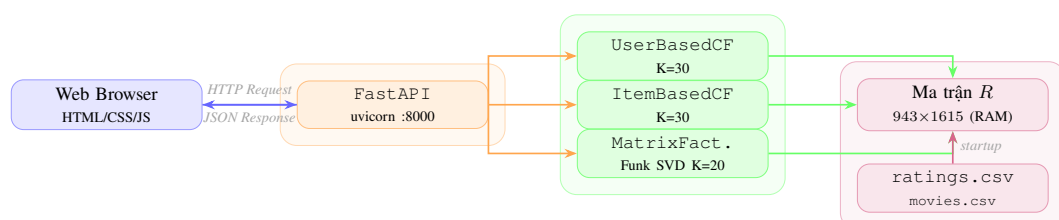
3.3.1 Kiến trúc tổng thể

Hệ thống được thiết kế theo mô hình *Client-Server* với ba lớp (Hình 3.2):

Lớp dữ liệu (Data Layer): File CSV được parse từ *u.data* và *u.item*. Ma trận *user-item* được xây dựng trong bộ nhớ RAM khi khởi động.

Lớp mô hình (Model Layer): Ba class Python: *UserBasedCF*, *ItemBasedCF*, *MatrixFactorization*. Tất cả được huấn luyện một lần khi khởi động và lưu trong bộ nhớ để phục vụ dự đoán *online* nhanh.

Lớp trình bày (Presentation Layer): FastAPI cung cấp RESTful API. Frontend HTML/CSS/JS gọi API và hiển thị kết quả.



Hình 3.2: Kiến trúc ba lớp của hệ thống **CineRec** và luồng dữ liệu khi xử lý một yêu cầu gọi ý

3.3.2 Thiết kế API

Bảng 3.1: Danh sách API endpoints

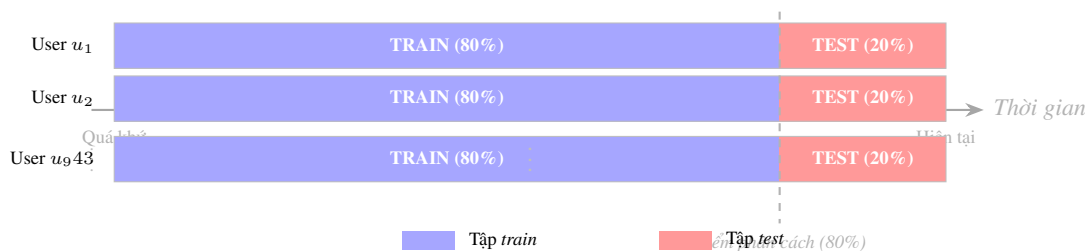
Endpoint	Method	Chức năng
GET /	GET	Trả về giao diện web (<code>index.html</code>)
GET /api/stats	GET	Thông kê dataset + <i>metrics</i> mô hình
GET /api/recommend/{id}	GET	Gợi ý phim cho <code>user_id</code> (params: method, n)
GET /api/health	GET	Kiểm tra trạng thái server

3.3.3 Cấu trúc dự án

```
recsys_ml100k/
+-- data/
|   +-- u.data      +-- u.item
|   +-- ratings.csv +-- movies.csv
+-- static/index.html
+-- models.py       (UserBasedCF, ItemBasedCF, MatrixFactorization)
+-- app.py          (FastAPI backend)
+-- run.py          (uvicorn launcher)
+-- demo_console.py (terminal demo)
```

3.4 Chiến lược chia tập Train/Test

Đồ án áp dụng **temporal split**: với mỗi người dùng, 80% *rating* sớm nhất làm *train*, 20% muộn nhất làm *test* (Hình 3.3). Chiến lược này phản ánh đúng kịch bản thực tế và tránh *data leakage*.



Hình 3.3: Temporal split theo từng người dùng: 80% *rating* sớm nhất làm *train*, 20% muộn nhất làm *test*. Mỗi người dùng được chia độc lập theo mốc thời gian của chính họ.

Bảng 3.2: Kết quả chia tập train/test (temporal split 80/20)

Tập	Số <i>rating</i>	Tỷ lệ	Mục đích
Train	≈80.000	≈80%	Huấn luyện mô hình
Test	≈20.000	≈20%	Đánh giá mô hình

3.5 Môi trường thực nghiệm

Bảng 3.3: Môi trường thực nghiệm

Thành phần	Thông số
Hệ điều hành	Windows 10+ / Ubuntu 22.04 LTS
CPU	Intel Core i5 (không yêu cầu GPU)
RAM	8 GB DDR4 (tối thiểu 4 GB)
Python	3.10+
Thư viện chính	NumPy 1.24, Pandas 2.0, FastAPI 0.110, Uvicorn 0.27
Thời gian khởi động	≈45–90 giây (huấn luyện 3 mô hình)

3.6 Cài đặt chi tiết các mô hình

3.6.1 User-based Collaborative Filtering

Mô hình được cài đặt với các bước chính: (1) tính vector *mean-centered* cho từng user; (2) tính ma trận tương đồng cosine 943×943 bằng phép nhân ma trận được vector hóa (*vectorized*) thay vì vòng lặp Python; (3) hàm `predict` tra cứu K hàng xóm tốt nhất và tính trung bình có trọng số.

```

1 mask      = (R_train != 0).astype(np.float32)
2 user_means = R_train.sum(1) / mask.sum(1).clip(min=1)
3 R_centered = (R_train - user_means[:, None]) * mask
4 norms      = np.linalg.norm(R_centered, axis=1,
5                             keepdims=True).clip(min=1e-9)
6 R_norm      = R_centered / norms          # (943, 1615)
7 sim_user    = R_norm @ R_norm.T          # (943, 943)
8 np.fill_diagonal(sim_user, 0)           # loại sim(u,u)

```

Listing 3.1: Tính ma trận tương đồng user-user (vectorized)

3.6.2 Item-based Collaborative Filtering

Sử dụng *adjusted cosine similarity* — trừ *user mean* trước khi tính cosine theo chiều user (transpose ma trận). Ma trận tương đồng item có kích thước 1.615×1.615 .

```

1 R_adj      = (R_train - user_means[:, None]) * mask
2 R_T        = R_adj.T                      # (1615, 943)
3 norms      = np.linalg.norm(R_T, axis=1,
4                             keepdims=True).clip(1e-9)
5 sim_item    = (R_T / norms) @ (R_T / norms).T # (1615, 1615)
6 np.fill_diagonal(sim_item, 0)

```

Listing 3.2: *Adjusted cosine similarity* cho Item-based CF

3.6.3 Matrix Factorization (Funk SVD)

Mô hình được cài đặt hoàn toàn từ đầu (*from scratch*) với class Python độc lập. Vòng lặp SGD xử lý từng *rating* trong mỗi *epoch*. Dữ liệu được xáo trộn ngẫu nhiên đầu mỗi *epoch* để tránh *bias* thứ tự.

```

1 for epoch in range(1, n_epochs + 1):
2     np.random.shuffle(rows)          # shuffle mọi epoch
3     for u, i, r_ui in rows:
4         r_hat = mu + bu[u] + bi[i] + P[u] @ Q[i]
5         e      = r_ui - r_hat        # residual
6         p_old  = P[u].copy()
7         P[u] += alpha * (e * Q[i] - lambda_ * P[u])
8         Q[i] += alpha * (e * p_old - lambda_ * Q[i])
9         bu[u] += alpha * (e          - lambda_ * bu[u])
10        bi[i] += alpha * (e          - lambda_ * bi[i])

```

Listing 3.3: Vòng lặp SGD cho Matrix Factorization (Funk SVD)

3.7 Kết quả đánh giá mô hình

3.7.1 MAE và RMSE

Bảng 3.4: Kết quả MAE và RMSE trên tập *test* (temporal split 80/20)

Mô hình	MAE	RMSE	Coverage	So với Baseline
Baseline (Global Mean)	$\approx 0,945$	$\approx 1,126$	100%	—
User-based CF ($K = 30$)	0,7746	0,9863	$\approx 98\%$	-12,4%
Item-based CF ($K = 30$)	0,7996	1,0211	$\approx 97\%$	-9,3%
Matrix Factorization	0,7706	0,9758	100%	-13,3%

Matrix Factorization đạt kết quả tốt nhất với $MAE = 0,7706$ và $RMSE = 0,9758$, cải thiện 13,3% $RMSE$ so với *baseline*. *User-based CF* đứng thứ hai với $RMSE = 0,9863$. *Item-based CF* cho $RMSE$ cao hơn (1,0211) trong thực nghiệm này, có thể do tỷ lệ *sparsity* cao (93,7%) làm giảm chất lượng tương đồng item.

3.7.2 Precision@K và Recall@K

Bảng 3.5: Kết quả Precision@10 và Recall@10 (tập con 200 users, ngưỡng *relevant* = rating ≥ 4)

Mô hình	Precision@10	Recall@10	Nhận xét
User-based CF ($K = 30$)	0,0047	0,0034	Thấp do <i>sparse K-NN</i>
Item-based CF ($K = 30$)	0,0152	0,0092	Trung bình
Matrix Factorization	0,0518	0,0484	Tốt nhất

Matrix Factorization vượt trội rõ rệt với Precision@10 = 0,0518 (gấp ≈ 11 lần *User-based CF*) và Recall@10 = 0,0484. Điều này cho thấy MF không chỉ dự đoán *rating* chính xác hơn mà còn tạo ra danh sách gợi ý phù hợp hơn với sở thích thực tế của người dùng.

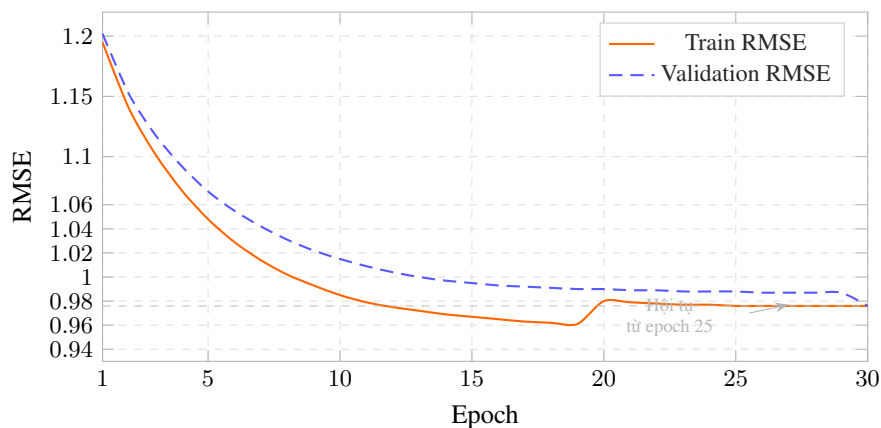
3.7.3 Điều chỉnh siêu tham số và đường cong hội tụ

Thực hiện *grid search* trên một phần dữ liệu để tìm siêu tham số tối ưu cho *Matrix Factorization*:

Bảng 3.6: Kết quả điều chỉnh siêu tham số MF

Siêu tham số	Giá trị thử	Tốt nhất	Tác động
K (latent factors)	10, 20, 50	20	$K = 20$ cân bằng bias/variance
α (learning rate)	0,005; 0,010	0,005	α nhỏ ổn định hơn
λ (regularization)	0,01; 0,02	0,02	Giảm <i>overfitting</i> tốt
Số <i>epochs</i>	10, 20, 30	30	Hội tụ ổn định sau epoch 25

Hình 3.4 minh họa đường cong $RMSE$ theo *epoch* của mô hình MF với bộ siêu tham số tối ưu ($K = 20$, $\alpha = 0,005$, $\lambda = 0,02$, 30 *epochs*). Tập *train* hội tụ đều đặn, tập *validation* giảm song song và ổn định từ *epoch* ≈ 25 , chứng tỏ mô hình không bị *overfitting*.



Hình 3.4: Đường cong $RMSE$ theo *epoch* của mô hình *Matrix Factorization* ($K = 20$, $\alpha = 0,005$, $\lambda = 0,02$). *Train* và *Validation* hội tụ song song, không có dấu hiệu *overfitting*.

3.8 Ví dụ kết quả gợi ý

Bảng 3.7: Top-10 gợi ý cho User 1 (Matrix Factorization, $K = 20$)

#	Tên phim	Thể loại chính	Pred. Rating
1	Lawrence of Arabia (1962)	Drama War	5,00
2	Wrong Trousers, The (1993)	Animation Comedy	4,93
3	Close Shave, A (1995)	Animation Comedy	4,91
4	Good Will Hunting (1997)	Drama Romance	4,80
5	Gandhi (1982)	Biography Drama	4,76
6	L.A. Confidential (1997)	Crime Drama	4,74
7	Casablanca (1942)	Drama Romance War	4,73
8	Titanic (1997)	Drama Romance	4,72
9	Butch Cassidy & Sundance Kid (1969)	Action Western	4,67
10	Secrets & Lies (1996)	Drama	4,56

3.9 Giao diện demo hệ thống

3.9.1 Hướng dẫn cài đặt và chạy

1. Cài đặt các thư viện cần thiết:

```
1 pip install fastapi uvicorn pandas numpy scikit-learn
```

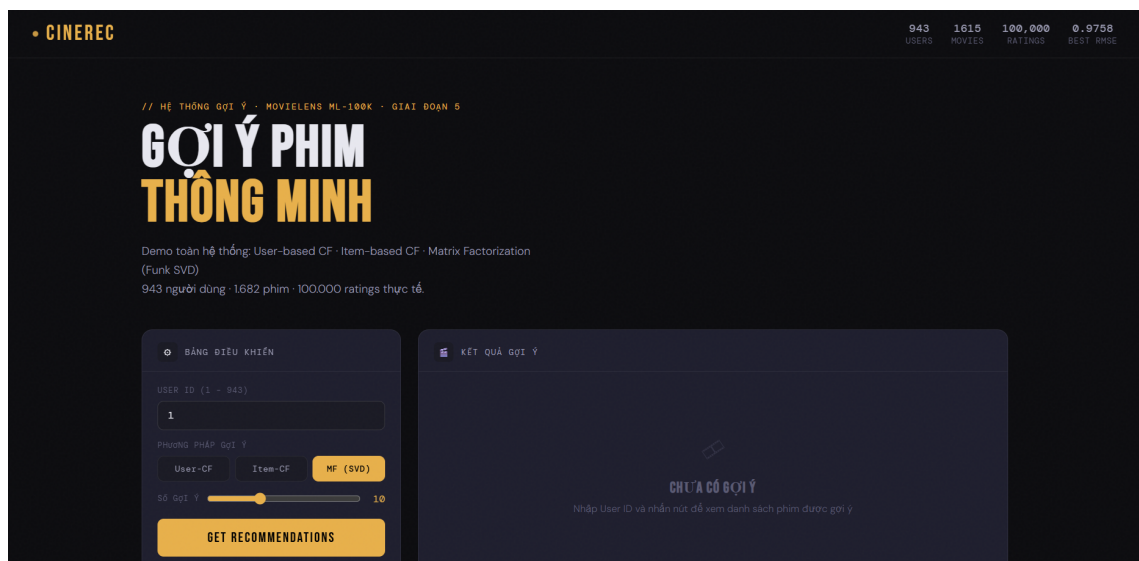
2. Di chuyển vào thư mục dự án và khởi động:

```
1 cd recsys_ml100k
2 python run.py
```

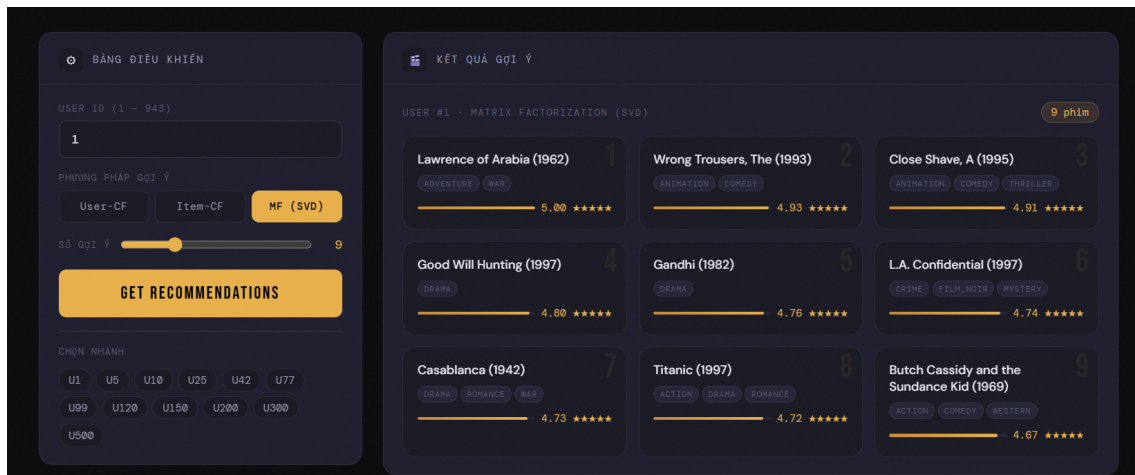
3. Mở trình duyệt và truy cập: `http://localhost:8000`

3.9.2 Mô tả giao diện

Giao diện web **CineRec** được thiết kế với phong cách tối (*dark theme*). Hình 3.5 minh họa tổng quan giao diện và Hình 3.6 minh họa kết quả gợi ý dạng card.



Hình 3.5: Tổng quan giao diện **CineRec** (*dark theme*): header thống kê hệ thống, bảng điều khiển bên trái (nhập User ID, chọn phương pháp, điều chỉnh số gợi ý), panel kết quả bên phải.



Hình 3.6: Kết quả gợi ý *Top-10* cho User 1 (MF SVD) hiển thị dạng card với thanh điểm dự đoán và số sao trực quan (khớp với Bảng 3.7), kèm lịch sử xem với màu sắc phân biệt mức *rating* (xanh lá = cao, đỏ = thấp).

3.9.3 Kiểm tra hoạt động toàn hệ thống

Quá trình kiểm tra *end-to-end* xác nhận:

- Server khởi động thành công, huấn luyện 3 mô hình trong $\approx 45-90$ giây.
- API `/api/stats` trả về đúng thông kê dataset và *metrics* ba mô hình.
- API `/api/recommend/{user_id}` trả về đúng danh sách Top-N kèm lịch sử xem.
- Frontend gọi API, nhận JSON và render giao diện đúng yêu cầu. Thử nghiệm với nhiều user (1, 42, 200, 500, 943) — hệ thống hoạt động ổn định.
- Xử lý lỗi: nhập User ID ngoài phạm vi $[1, 943]$ $\rightarrow$ thông báo lỗi rõ ràng trên giao diện (HTTP 404 kèm message tiếng Việt).

CHƯƠNG 4. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

4.1 Kết quả đạt được

Sau quá trình nghiên cứu và thực hiện, đồ án đã đạt được đầy đủ các mục tiêu đề ra tại mục 1.2.

4.1.1 Về lý thuyết

- Nắm vững nền tảng lý thuyết về hệ thống gợi ý, phân biệt rõ ba phương pháp tiếp cận: *Content-based Filtering*, *Collaborative Filtering* và *Hybrid*.
- Hiểu sâu cơ chế hoạt động của *User-based CF* và *Item-based CF*, bao gồm *mean-centered cosine similarity* và *adjusted cosine similarity*.
- Nắm vững thuật toán *Matrix Factorization* (Funk SVD), hàm mất mát với *regularization L2* và cơ chế SGD cập nhật tham số.
- Hiểu và áp dụng đúng bốn chỉ số đánh giá: MAE, RMSE, $\text{Precision}@K$, $\text{Recall}@K$; phân biệt khi nào dùng chỉ số nào.

4.1.2 Về kỹ thuật

- Triển khai thành công ba mô hình CF *from scratch* bằng Python/NumPy, không phụ thuộc thư viện ML chuyên dụng (`Scikit-learn` chỉ được dùng để tính chỉ số MAE/RMSE).
- Áp dụng *vectorized computation* (nhân ma trận NumPy) để tính ma trận tương đồng hiệu quả, thay thế vòng lặp Python thuần túy.
- Cài đặt *temporal split* tránh *data leakage*. Thực hiện *grid search* tối ưu siêu tham số (K , α , λ).
- Xây dựng hệ thống *backend/frontend* hoàn chỉnh: FastAPI RESTful API + giao diện web *responsive dark theme*.

4.1.3 Về kết quả thực nghiệm

Bảng 4.1 tổng hợp kết quả thực nghiệm trên bộ dữ liệu MovieLens ml-100k (100.000 *ratings*, 943 users, 1.682 phim).

Bảng 4.1: Tổng hợp kết quả thực nghiệm trên MovieLens ml-100k

Mô hình	MAE	RMSE	P@10	R@10	Coverage	Xếp hạng
Baseline (Mean)	$\approx 0,945$	$\approx 1,126$	—	—	100%	—
User-based CF ($K = 30$)	0,7746	0,9863	0,0047	0,0034	$\approx 98\%$	2
Item-based CF ($K = 30$)	0,7996	1,0211	0,0152	0,0092	$\approx 97\%$	3
Matrix Factorization	0,7706	0,9758	0,0518	0,0484	100%	1

Matrix Factorization (Funk SVD, $K = 20$) đạt kết quả tốt nhất trên tất cả các chỉ số: RMSE = 0,9758 (cải thiện 13,3% so với *baseline*), $\text{Precision}@10 = 0,0518$ (gấp ≈ 11 lần *User-based CF*) và *coverage* 100%. *User-based CF* đứng thứ hai với RMSE = 0,9863, trong khi *Item-based CF* bị ảnh hưởng nhiều hơn bởi độ thưa thớt 93,7% của dữ liệu.

4.2 Hạn chế của đề tài

Bên cạnh những kết quả đạt được, đề tài còn tồn tại một số hạn chế:

- Cold start problem** (vấn đề khởi động nguội): Các mô hình CF và MF không thể gợi ý cho người dùng mới (chưa có *rating* nào) hoặc phim mới (chưa được ai đánh giá). Đây là hạn chế cơ bản của *Collaborative Filtering*. Hệ thống hiện xử lý bằng cách *fallback* về *rating* trung bình, nhưng chưa giải

quyết triệt để.

- **Scalability** (khả năng mở rộng): Ma trận tương đồng 943×943 (UBCF) và 1.615×1.615 (IBCF) còn quản lý được trong RAM. Tuy nhiên với dataset lớn hơn (hàng triệu users/items), cần các giải pháp *Approximate Nearest Neighbors* (ANN) như Faiss hay HNSW.
- **SGD chưa tối ưu hóa**: Vòng lặp SGD thuần Python (≈ 80.000 iterations/epoch) còn chậm do không được vector hóa hoàn toàn. Có thể tăng tốc đáng kể bằng Cython hoặc PyTorch.
- **Chỉ dùng rating tường minh**: Mô hình chưa khai thác dữ liệu ngầm định (*implicit feedback*) như lượt xem, thời gian xem, lịch sử tìm kiếm — vốn phong phú hơn nhiều trong thực tế.
- **Chưa có Content-based**: Thông tin nội dung phim (thể loại, đạo diễn, diễn viên) chưa được tận dụng để bổ sung đặc trưng cho mô hình hay giải quyết *cold start*.

4.3 Hướng phát triển tương lai

Dựa trên những kết quả và hạn chế trên, các hướng phát triển tiếp theo được đề xuất:

1. **Neural Collaborative Filtering (NCF)**: Thay thế tích vô hướng $p_u \cdot q_i$ bằng mạng neural sâu để học được tương tác phi tuyến phức tạp giữa user và item. NCF đã được chứng minh vượt trội so với MF truyền thống trên nhiều *benchmark* [8].
2. **Hybrid Recommender System**: Kết hợp *Collaborative Filtering* với *Content-based Filtering* (thông tin thể loại, đạo diễn, diễn viên) để giải quyết *cold start problem* và cải thiện độ đa dạng gợi ý.
3. **Implicit Feedback Model**: Áp dụng các mô hình như BPR (*Bayesian Personalized Ranking*) hoặc ALS (*Alternating Least Squares*) trên dữ liệu ngầm định thay vì chỉ *rating* tường minh.
4. **Đánh giá đa chiều**: Bổ sung thêm các chỉ số NDCG@K (*Normalized Discounted Cumulative Gain*), MRR (*Mean Reciprocal Rank*) và *Coverage* để đánh giá toàn diện hơn chất lượng danh sách gợi ý.
5. **Triển khai production**: Tích hợp *caching* (Redis), *authentication*, *logging* và *containerization* (Docker) để triển khai hệ thống trong môi trường thực tế.

TÀI LIỆU THAM KHẢO

- [1] P. Resnick, N. Iacovou, M. Suchak, P. Bergstrom, and J. Riedl, “GroupLens: An open architecture for collaborative filtering of netnews,” in *Proceedings of the 1994 ACM Conference on Computer Supported Cooperative Work (CSCW '94)*, ACM, 1994, pp. 175–186.
- [2] F. Ricci, L. Rokach, B. Shapira, and P. B. Kantor, *Recommender Systems Handbook*. New York: Springer, 2011.
- [3] B. Sarwar, G. Karypis, J. Konstan, and J. Riedl, “Item-based collaborative filtering recommendation algorithms,” in *Proceedings of the 10th International Conference on World Wide Web (WWW '01)*, ACM, 2001, pp. 285–295.
- [4] Y. Koren, R. Bell, and C. Volinsky, “Matrix factorization techniques for recommender systems,” *Computer*, vol. 42, no. 8, pp. 30–37, 2009.
- [5] S. Funk, *Netflix update: Try this at home*, <https://sifter.org/~simon/journal/20061211.html> [Truy cập: 15/06/2026], 2006.
- [6] J. L. Herlocker, J. A. Konstan, L. G. Terveen, and J. T. Riedl, “Evaluating collaborative filtering recommender systems,” *ACM Transactions on Information Systems (TOIS)*, vol. 22, no. 1, pp. 5–53, 2004.
- [7] F. M. Harper and J. A. Konstan, “The MovieLens datasets: History and context,” *ACM Transactions on Interactive Intelligent Systems (TiiS)*, vol. 5, no. 4, pp. 1–19, 2015.
- [8] X. He, L. Liao, H. Zhang, L. Nie, X. Hu, and T.-S. Chua, “Neural collaborative filtering,” in *Proceedings of the 26th International Conference on World Wide Web (WWW '17)*, ACM, 2017, pp. 173–182.

PHỤ LỤC

A. MÃ NGUỒN HỆ THỐNG CINEREC

Phụ lục này trình bày toàn bộ mã nguồn cốt lõi của hệ thống **CineRec**. Mã nguồn được tổ chức thành bốn file chính: (1) `models.py` chứa lõi thuật toán, (2) `app.py` là backend FastAPI, (3) `run.py` là script khởi động, và (4) `demo_console.py` là giao diện terminal. Toàn bộ mã nguồn cũng có thể được tải về tại kho lưu trữ của đề tài.

A.1 File `models.py` — Lõi thuật toán

```
1  """
2  models.py - Recommendation Engine (MovieLens ml-100k)
3  User-based CF | Item-based CF | Matrix Factorization (Funk SVD)
4  """
5  import numpy as np
6  import pandas as pd
7  from sklearn.metrics import mean_absolute_error, mean_squared_error
8
9  # -- DATA -----
10
11 def load_data(ratings_path="data/ratings.csv",
12              movies_path="data/movies.csv"):
13     df_r = pd.read_csv(ratings_path)
14     df_m = pd.read_csv(movies_path)
15     info = df_m.set_index("movieId") [{"title", "genres"}
16                                         ].to_dict("index")
17     return df_r, df_m, info
18
19
20 def split_data(df, test_size=0.20):
21     """Temporal split: moi user giu 20% rating cuoi lam test."""
22     df = df.sort_values(["userId", "timestamp"])
23     train, test = [], []
24     for _, g in df.groupby("userId"):
25         n = max(1, int(len(g) * test_size))
26         train.append(g.iloc[:-n])
27         test.append(g.iloc[-n:])
28     return (pd.concat(train).reset_index(drop=True),
29           pd.concat(test).reset_index(drop=True))
30
31
32 def build_matrix(df_train):
33     piv = df_train.pivot_table(
34         index="userId", columns="movieId",
35         values="rating")
36     R = piv.fillna(0).values.astype(np.float32)
37     uid2i = {u: i for i, u in enumerate(piv.index)}
38     mid2j = {m: j for j, m in enumerate(piv.columns)}
39     return R, uid2i, mid2j, list(piv.index), list(piv.columns)
40
41
42 def evaluate(df_test, predict_fn, uid2i, mid2j):
43     preds, actuals = [], []
44     for _, row in df_test.iterrows():
45         u, m = int(row["userId"]), int(row["movieId"])
46         if u in uid2i and m in mid2j:
47             preds.append(predict_fn(u, m))
48             actuals.append(row["rating"])
49     if not preds:
50         return {"mae": None, "rmse": None, "coverage": 0}
51     return {
```

```

52     "mae": round(mean_absolute_error(actuals, preds), 4),
53     "rmse": round(float(np.sqrt(
54         mean_squared_error(actuals, preds))), 4),
55     "coverage": round(len(preds)/len(df_test)*100, 1),
56 }
57
58
59 def precision_recall_at_k(df_test, recommend_fn,
60     uid2i, K=10, threshold=4.0):
61     precisions, recalls = [], []
62     for uid in uid2i:
63         relevant = set(df_test[
64             (df_test["userId"]==uid) &
65             (df_test["rating"]>=threshold)][["movieId"])
66         if not relevant:
67             continue
68         recs = [mid for mid, _ in recommend_fn(uid, n=K)]
69         hits = len(set(recs) & relevant)
70         precisions.append(hits / K)
71         recalls.append(hits / len(relevant))
72     return {
73         f"precision@{K}": round(float(np.mean(precisions)), 4)
74         if precisions else 0,
75         f"recall@{K}": round(float(np.mean(recalls)), 4)
76         if recalls else 0,
77     }
78
79 # -- USER-BASED CF -----
80
81 class UserBasedCF:
82     def __init__(self, K=30):
83         self.K = K
84
85     def fit(self, R, uid2i, mid2j, user_ids, movie_ids,
86         df_train):
87         self.R = R; self.uid2i = uid2i; self.mid2j = mid2j
88         self.user_ids = user_ids; self.movie_ids = movie_ids
89         self.df_train = df_train
90         mask = (R != 0).astype(np.float32)
91         self.umeans = np.true_divide(
92             R.sum(1), mask.sum(1).clip(min=1))
93         Rc = (R - self.umeans[:, None]) * mask
94         nrm = np.linalg.norm(Rc, axis=1,
95             keepdims=True).clip(min=1e-9)
96         Rn = Rc / nrm
97         self.sim = Rn @ Rn.T
98         np.fill_diagonal(self.sim, 0)
99
100     def predict(self, user_id, movie_id):
101         if user_id not in self.uid2i \
102             or movie_id not in self.mid2j:
103             return float(self.umeans.mean())
104         u = self.uid2i[user_id]
105         j = self.mid2j[movie_id]
106         sims = self.sim[u].copy()
107         sims[self.R[:, j] == 0] = 0
108         tk = np.argsort(sims)[-1:][self.K]
109         ts = sims[tk]; pos = ts > 0
110         if not pos.any():
111             return float(self.umeans[u])
112         tk, ts = tk[pos], ts[pos]

```

```

113         pred = (self.umeans[u]
114                 + np.dot(ts, self.R[tk,j]-self.umeans[tk])
115                 / ts.sum())
116         return float(np.clip(pred, 1, 5))
117
118     def recommend(self, user_id, n=10):
119         if user_id not in self.uid2i:
120             return []
121         seen = set(self.df_train[
122                     self.df_train["userId"]==user_id]["movieId"])
123         sc = [(m, self.predict(user_id, m))
124               for m in self.movie_ids if m not in seen]
125         return sorted(sc, key=lambda x: x[1],
126                       reverse=True)[:n]
127
128     # -- ITEM-BASED CF -----
129
130     class ItemBasedCF:
131         def __init__(self, K=30):
132             self.K = K
133
134         def fit(self, R, uid2i, mid2j, user_ids, movie_ids,
135               df_train):
136             self.R = R; self.uid2i = uid2i; self.mid2j = mid2j
137             self.user_ids = user_ids; self.movie_ids = movie_ids
138             self.df_train = df_train
139             mask = (R != 0).astype(np.float32)
140             umean = np.true_divide(R.sum(1),
141                                   mask.sum(1).clip(min=1))
142             self.umeans = umean
143             Radj = (R - umean[:, None]) * mask
144             RT = Radj.T
145             nrm = np.linalg.norm(RT, axis=1,
146                                  keepdims=True).clip(min=1e-9)
147             RTn = RT / nrm
148             self.sim = RTn @ RTn.T
149             np.fill_diagonal(self.sim, 0)
150
151         def predict(self, user_id, movie_id):
152             if user_id not in self.uid2i \
153                or movie_id not in self.mid2j:
154                 return float(self.umeans.mean())
155             u = self.uid2i[user_id]
156             j = self.mid2j[movie_id]
157             sims = self.sim[j].copy()
158             sims[self.R[u, :] == 0] = 0
159             sims[j] = 0
160             tk = np.argsort(sims)[::-1][:self.K]
161             ts = sims[tk]; pos = ts > 0
162             if not pos.any():
163                 return float(self.umeans[u])
164             tk, ts = tk[pos], ts[pos]
165             return float(np.clip(
166                 np.dot(ts, self.R[u, tk]) / ts.sum(), 1, 5))
167
168     def recommend(self, user_id, n=10):
169         if user_id not in self.uid2i:
170             return []
171         seen = set(self.df_train[
172                     self.df_train["userId"]==user_id]["movieId"])
173         sc = [(m, self.predict(user_id, m))

```

```

174         for m in self.movie_ids if m not in seen]
175     return sorted(sc, key=lambda x: x[1],
176                 reverse=True)[:n]
177
178 # -- MATRIX FACTORIZATION (FUNK SVD + SGD) -----
179
180 class MatrixFactorization:
181     def __init__(self, K=20, alpha=0.005,
182                 lambda_=0.02, n_epochs=30, seed=42):
183         self.K = K; self.alpha = alpha
184         self.lambda_ = lambda_
185         self.n_epochs = n_epochs; self.seed = seed
186         self.train_loss = []; self.val_loss = []
187
188     def fit(self, df_train, n_users, n_items,
189            uid2i, mid2j, df_val=None):
190         np.random.seed(self.seed)
191         self.uid2i = uid2i; self.mid2j = mid2j
192         self.df_train = df_train
193         self.P = np.random.normal(
194             0, 0.1, (n_users, self.K)).astype(np.float32)
195         self.Q = np.random.normal(
196             0, 0.1, (n_items, self.K)).astype(np.float32)
197         self.bu = np.zeros(n_users, dtype=np.float32)
198         self.bi = np.zeros(n_items, dtype=np.float32)
199         self.mu = float(df_train["rating"].mean())
200         rows = np.array([
201             (uid2i[int(r.userId)],
202             mid2j[int(r.movieId)], r.rating)
203             for r in df_train.itertuples()
204             if int(r.userId) in uid2i
205             and int(r.movieId) in mid2j
206         ], dtype=np.float32)
207         for ep in range(1, self.n_epochs + 1):
208             np.random.shuffle(rows)
209             sq = 0.0
210             for u, i, rui in rows:
211                 u, i = int(u), int(i)
212                 e = rui - (self.mu + self.bu[u]
213                           + self.bi[i]
214                           + self.P[u] @ self.Q[i])
215                 sq += e * e
216                 pu = self.P[u].copy()
217                 self.P[u] += self.alpha * (
218                     e * self.Q[i] - self.lambda_ * self.P[u])
219                 self.Q[i] += self.alpha * (
220                     e * pu - self.lambda_ * self.Q[i])
221                 self.bu[u] += self.alpha * (
222                     e - self.lambda_ * self.bu[u])
223                 self.bi[i] += self.alpha * (
224                     e - self.lambda_ * self.bi[i])
225                 self.train_loss.append(
226                     float(np.sqrt(sq / len(rows))))
227                 if df_val is not None:
228                     self.val_loss.append(self._rmse(df_val))
229
230     def predict(self, user_id, movie_id):
231         if user_id not in self.uid2i \
232            or movie_id not in self.mid2j:
233             return self.mu
234         u = self.uid2i[user_id]

```



```

235         i = self.mid2j[movie_id]
236         return float(np.clip(
237             self.mu + self.bu[u] + self.bi[i]
238             + self.P[u] @ self.Q[i], 1, 5))
239
240     def _rmse(self, df):
241         e = [(r.rating - self.predict(
242             int(r.userId), int(r.movieId)))**2
243             for r in df.itertuples()]
244         if int(r.userId) in self.uid2i
245         and int(r.movieId) in self.mid2j]
246         return float(np.sqrt(np.mean(e))) \
247             if e else float("nan")
248
249     def recommend(self, user_id, n=10):
250         if user_id not in self.uid2i:
251             return []
252         seen = set(self.df_train[
253             self.df_train["userId"]==user_id]["movieId"])
254         sc = [(m, self.predict(user_id, m))
255              for m in self.mid2j if m not in seen]
256         return sorted(sc, key=lambda x: x[1],
257                       reverse=True)[:n]
258
259 # -- FACADE -----
260
261 class RecommendationSystem:
262     def __init__(self,
263                 ratings_path="data/ratings.csv",
264                 movies_path="data/movies.csv"):
265         print(" [1/4] Doc du lieu ml-100k...")
266         self.df_r, self.df_m, self.info = load_data(
267             ratings_path, movies_path)
268         self.df_train, self.df_test = split_data(self.df_r)
269         (self.R, self.uid2i, self.mid2j,
270          self.uids, self.mids) = build_matrix(self.df_train)
271         nu, ni = len(self.uid2i), len(self.mid2j)
272
273         print(" [2/4] User-based CF...")
274         self.ubcf = UserBasedCF(K=30)
275         self.ubcf.fit(self.R, self.uid2i, self.mid2j,
276                       self.uids, self.mids, self.df_train)
277
278         print(" [3/4] Item-based CF...")
279         self.ibcf = ItemBasedCF(K=30)
280         self.ibcf.fit(self.R, self.uid2i, self.mid2j,
281                       self.uids, self.mids, self.df_train)
282
283         print(" [4/4] Matrix Factorization (30 epochs)...")
284         self.mf = MatrixFactorization(
285             K=20, alpha=0.005, lambda_=0.02, n_epochs=30)
286         self.mf.fit(self.df_train, nu, ni,
287                     self.uid2i, self.mid2j, self.df_test)
288
289         print(" Danh gia mo hinh...")
290         self.metrics = {
291             "ubcf": evaluate(self.df_test,
292                             self.ubcf.predict, self.uid2i, self.mid2j),
293             "ibcf": evaluate(self.df_test,
294                             self.ibcf.predict, self.uid2i, self.mid2j),
295             "mf": evaluate(self.df_test,

```

```

296         self.mf.predict, self.uid2i, self.mid2j),
297     }
298     sample_uids = list(self.uid2i.keys())[:200]
299     df_test_sub = self.df_test[
300         self.df_test["userId"].isin(sample_uids)]
301     uid2i_sub = {u: i for u, i in self.uid2i.items()
302                  if u in sample_uids}
303     for name, model in [("ubcf", self.ubcf),
304                        ("ibcf", self.ibcf),
305                        ("mf", self.mf)]:
306         pr = precision_recall_at_k(
307             df_test_sub, model.recommend,
308             uid2i_sub, K=10)
309         self.metrics[name].update(pr)
310     print(" San sang!\n")
311
312     def recommend(self, user_id: int,
313                  method="mf", n=10):
314         model = {"ubcf": self.ubcf,
315                 "ibcf": self.ibcf,
316                 "mf": self.mf}[method]
317         raw = model.recommend(user_id, n=n)
318         result = []
319         for mid, score in raw:
320             inf = self.info.get(mid, {})
321             result.append({
322                 "movieId": mid,
323                 "title": inf.get(
324                     "title", f"Movie {mid}"),
325                 "genres": inf.get(
326                     "genres", "Unknown"),
327                 "predicted_rating": round(score, 2),
328             })
329         return result
330
331     def history(self, user_id: int, n=8):
332         rows = (self.df_train[
333                 self.df_train["userId"] == user_id]
334                 .sort_values("rating", ascending=False)
335                 .head(n))
336         out = []
337         for r in rows.itertuples():
338             inf = self.info.get(int(r.movieId), {})
339             out.append({
340                 "movieId": int(r.movieId),
341                 "title": inf.get(
342                     "title", f"Movie {r.movieId}"),
343                 "genres": inf.get("genres", ""),
344                 "rating": int(r.rating),
345             })
346         return out
347
348     def stats(self):
349         return {
350             "n_users": len(self.uid2i),
351             "n_movies": len(self.mid2j),
352             "n_ratings": len(self.df_r),
353             "n_train": len(self.df_train),
354             "n_test": len(self.df_test),
355             "sparsity": round(
356                 (1 - len(self.df_r)

```

```

357         / (len(self.uid2i)*len(self.mid2j))
358     )*100, 2),
359     "avg_rating": round(
360         float(self.df_r["rating"].mean()), 2),
361     "metrics": self.metrics,
362     "mf_train_loss": self.mf.train_loss,
363     "mf_val_loss": self.mf.val_loss,
364 }
365
366 @property
367 def all_users(self):
368     return sorted(self.uid2i.keys())

```

Listing A.1: models.py – User-based CF / Item-based CF / Matrix Factorization / RecommendationSystem facade

A.2 File app.py — FastAPI Backend

```

1  """
2  app.py - FastAPI Backend
3  Chạy: uvicorn app:app --reload --port 8000
4  """
5  from fastapi import FastAPI, HTTPException
6  from fastapi.staticfiles import StaticFiles
7  from fastapi.responses import HTMLResponse, FileResponse
8  import time
9  from models import RecommendationSystem
10
11 app = FastAPI(title="CineRec - ML100K", version="1.0")
12
13 print("\n" + "="*55)
14 print("  CINEREC - MovieLens ml-100k   |   Khoi dong...")
15 print("="*55)
16 t0 = time.time()
17 rs = RecommendationSystem(
18     "data/ratings.csv", "data/movies.csv")
19 print(f"  Khoi dong xong trong {time.time()-t0:.1f}s")
20
21 app.mount("/static",
22         StaticFiles(directory="static"), name="static")
23
24 @app.get("/", response_class=HTMLResponse)
25 async def root():
26     return FileResponse("static/index.html")
27
28 @app.get("/api/stats")
29 async def get_stats():
30     return rs.stats()
31
32 @app.get("/api/users")
33 async def get_users():
34     return {"user_ids": rs.all_users}
35
36 @app.get("/api/recommend/{user_id}")
37 async def recommend(user_id: int,
38                     method: str = "mf", n: int = 10):
39     if user_id not in rs.uid2i:
40         raise HTTPException(
41             404, f"User {user_id} không tồn tại (1-943)")
42     if method not in ("ubcf", "ibcf", "mf"):
43         raise HTTPException(
44             400, "method phải là: ubcf | ibcf | mf")
45     recs = rs.recommend(user_id, method=method, n=n)

```

```

46     history = rs.history(user_id)
47     return {"user_id": user_id, "method": method,
48            "recommendations": recs, "history": history}
49
50 @app.get("/api/health")
51 async def health():
52     return {"status": "ok", "dataset": "ml-100k",
53            "users": rs.stats()["n_users"],
54            "movies": rs.stats()["n_movies"]}

```

Listing A.2: app.py – FastAPI backend, định nghĩa 4 endpoints

A.3 File run.py — Script khởi động

```

1  """
2  run.py - Khởi động CineRec Demo
3  Chạy: python run.py
4  Mô trình duyệt: http://localhost:8000
5  """
6  import subprocess, sys, os, time, webbrowser, threading
7
8  def open_browser():
9      time.sleep(3)
10     webbrowser.open("http://localhost:8000")
11
12  if __name__ == "__main__":
13     os.chdir(os.path.dirname(os.path.abspath(__file__)))
14     threading.Thread(
15         target=open_browser, daemon=True).start()
16     subprocess.run([
17         sys.executable, "-m", "uvicorn", "app:app",
18         "--host", "0.0.0.0", "--port", "8000", "--reload"
19     ])

```

Listing A.3: run.py – script khởi động nhanh bằng uvicorn, tự động mở trình duyệt